

Raw Material Knowledge Engine

Technical Report

MAERII Project Team

June 22, 2026

Raw Material Knowledge Engine

Technical Report

MAERII: Market Alignment Engine for Rural and Indigenous Innovation

June 22, 2026

Report Attribute	Details
Project Module	Tribal Raw Material Knowledge Engine
Project Domain	Minor Forest Produce, tribal livelihoods, AI-assisted data enrichment
Primary Output	Structured MFP knowledge base with evidence and relationship mapping
Source Dataset	TRIFED Minor Forest Produce list comprising 87 items
Prepared For	Technical review and project documentation
Prepared By	MAERII Project Team

Report Attribute	Details
------------------	---------

This report documents the design, methodology, implementation, and technical outcomes of the Raw Material Knowledge Engine developed under the MAERII project.

1 Table of Contents

Contents

1	Table of Contents	i
2	Executive Summary	1
3	Problem Statement	1
4	System Architecture and Methodology	1
4.1	Phase 1: Base Enrichment	1
4.2	Phase 2: Deep Enrichment	1
4.3	Data Flow	2
5	Technology Stack and Justification	2
6	Evidence and Provenance Model	3
7	Relationship Graph Model	3
8	Results and Statistics	4
9	Safeguards and Rollback Design	4
10	Challenges and System Refinements	4
11	Conclusion	5

2 Executive Summary

This technical report presents the development of the Raw Material Knowledge Engine, an intelligent data pipeline and knowledge graph designed to centralize and structure information on Minor Forest Produce (MFP). The system processes a base dataset of 87 MFP items and enriches it with structured attributes, provenance records, and relationship edges that can support later MAERII modules.

The engine was developed to convert fragmented and unstructured information into a usable knowledge layer. It combines structured seed data, web search, document retrieval, LLM-assisted extraction, validation, and graph-oriented relationship modeling. The resulting system provides a foundation for market demand analysis, product recommendation, quality assessment, and pricing intelligence.

3 Problem Statement

Information related to tribal raw materials and Minor Forest Produce is highly fragmented. Useful data on geographical availability, production quantities, processing skills, and value-added products is commonly scattered across government notifications, institutional PDFs, research papers, biodiversity portals, and isolated web pages.

This fragmentation creates a practical barrier for decision-making. Stakeholders may know that a raw material exists, but may not know where it is available, which communities or skill groups are associated with it, what products can be made from it, or how strongly each extracted claim is supported by evidence.

The objective of this project was to build an automated, AI-assisted system capable of extracting, structuring, validating, and interlinking fragmented raw material knowledge into a traceable and queryable dataset.

4 System Architecture and Methodology

The Raw Material Knowledge Engine follows a two-phase enrichment methodology. The first phase creates a structured baseline dataset from the seed MFP list. The second phase adds deeper evidence-backed attributes and graph relationships without disturbing the original enriched data.

4.1 Phase 1: Base Enrichment

The base enrichment pipeline ingests a seed list of 87 MFP items and standardizes core attributes. The primary fields extracted and normalized in this phase include scientific names, product descriptions, seasonality, shelf life, applicable states, artisan types, current products, potential value-added products, image URLs, and confidence scores.

This phase produces the core canonical dataset stored in `data/enriched_mfp_data.json`. The file serves as the primary structured knowledge base for the project and remains the protected baseline for later additive enrichment.

4.2 Phase 2: Deep Enrichment

The deep enrichment phase focuses on additional supply-side intelligence. It is designed to add information about quantitative availability, granular geography, tribal or production clusters, and graph-ready relationships.

For each MFP item, the system generates targeted search queries, retrieves relevant HTML and PDF documents, extracts structured evidence, and attaches the new information under a separate `deep_enrichment` object. This additive design prevents existing base-enrichment fields from being overwritten.

The deep enrichment phase supports the following operations:

1. Targeted search for procurement, production, collection, yield, district, cluster, Van Dhan, value addition, and livelihood signals.
2. Retrieval of HTML and PDF sources, including institutional and research documents.
3. LLM-assisted extraction of availability and geography fields.
4. Deterministic generation of relationship edges from existing and newly extracted fields.
5. Evidence linking through source URLs, snippets, source types, and confidence scores.

4.3 Data Flow

The technical data flow is summarized below.

Stage	Input	Process	Output
Seed Preparation	TRIFED MFP CSV	CSV parsing, MSP cleanup, state normalization	<code>seed_mfp_list.json</code>
Base Enrichment	Seed JSON	Search, fetch, LLM extraction, validation	<code>enriched_mfp_data.json</code>
Deep Retrieval	Enriched item records	Targeted search and document fetching	Source documents and snippets
Deep Extraction	Retrieved source text	Focused extraction of availability and geography	Additive <code>deep_enrichment</code> patch
Evidence Modeling	Extracted attributes	Evidence ID generation and source mapping	<code>deep_enrichment_evidence.json</code>
Relationship Modeling	Existing and deep fields	Graph edge construction	<code>material_relationships.json</code>

5 Technology Stack and Justification

The system uses a Python-based data engineering stack because Python provides mature libraries for web retrieval, document parsing, structured data processing, and LLM integration.

Component	Technology	Purpose
Programming Language	Python	Core pipeline implementation and orchestration
Search Provider	Serper API	Programmatic search over web pages, institutional sources, and PDFs
Web Scraping	<code>requests</code> , <code>beautifulsoup4</code>	HTML retrieval and text extraction
PDF Extraction	<code>pypdf</code>	Text extraction from reports, circulars, and research PDFs

Component	Technology	Purpose
LLM Providers	Groq, Gemini fallback	Structured extraction from unstructured source text
Data Storage	JSON files	Portable canonical dataset, evidence store, and graph sidecars
Relationship Model	Graph-ready JSON edges	Queryable relationships across materials, regions, skills, and products
Validation	Custom Python validators	Coverage checks, confidence review, referential integrity checks

The current implementation intentionally uses JSON outputs rather than introducing a database dependency at this stage. This keeps the system lightweight, auditable, and easy to test. The relationship sidecar is nevertheless graph-ready and can later be migrated into Neo4j or another graph database.

6 Evidence and Provenance Model

A central design principle of the Knowledge Engine is traceability. Extracted information is not treated as trustworthy merely because it was generated by an LLM. Each deep-enrichment attribute is linked to evidence through an `evidence_id`.

Each evidence record contains the material ID, attribute group, attribute name, extracted value, source URL, source type, supporting snippet, retrieval timestamp, and confidence score. This structure allows reviewers to trace any enriched value back to the source from which it was derived.

The evidence model supports the following goals:

- reducing hallucination risk in LLM-assisted extraction
- enabling human audit of uncertain values
- supporting future validation and confidence scoring
- preserving source transparency for downstream modules

7 Relationship Graph Model

The Knowledge Engine also produces graph-ready relationship edges. These edges represent structured links between a raw material and related entities such as states, districts, clusters, skills, current products, potential products, and material groups.

The relationship model is stored separately from the main dataset in `data/material_relationships.json`. This design keeps the canonical item file readable while giving future modules a normalized graph structure for querying and reasoning.

Edge Type	Meaning
<code>available_in</code>	Links a material to a state or district
<code>linked_to</code>	Links a material to a cluster or institutional grouping

Edge Type	Meaning
processed_by	Links a material to artisan or processing skills
used_for_product	Links a material to current products
could_enable_product	Links a material to potential value-added products
belongs_to_group	Links a material to its broader category

8 Results and Statistics

The pipeline execution generated a structured dataset and relationship layer from the 87-item MFP seed list.

Metric	Value
Total MFP items processed	87
Items successfully deep-enriched	86
Relationship edges extracted	1,259
Evidence records captured	1,502

The relationship edge distribution is summarized below.

Relationship Category	Count
Available in	426
Used for product	317
Could enable product	236
Processed by	177
Belongs to group	86
Linked to	17

9 Safeguards and Rollback Design

The deep enrichment implementation is designed to protect the existing `data/enriched_mfp_data.json` file. Existing top-level fields are not rewritten during deep enrichment. New data is added only under the `deep_enrichment` object for each targeted material.

Before any apply-mode run, the system creates a timestamped backup of the canonical dataset and sidecar files. Each run also records a manifest containing the run ID, mode, item IDs, batch size, resume index, output paths, backup paths, status, and errors.

Rollback is supported through a run ID. When rollback is triggered, the system restores the backed-up versions of the canonical file, evidence file, and relationship file using atomic JSON writes. This mechanism allows the project to test pilot enrichment safely before expanding to larger batches.

10 Challenges and System Refinements

Two major challenges emerged during development.

First, early search queries were too broad and keyword-heavy. This reduced search quality and often produced generic documents. The query strategy was refined to generate more natural, targeted search strings such as item-specific district, production, and value-addition queries.

Second, strict extraction prompts initially caused the LLM to omit useful geographic references unless the source text explicitly used labels such as “district” or “cluster.” The extraction strategy was adjusted to remain conservative while still allowing supported district and cluster signals to be captured.

These refinements improved the ability of the system to identify specific evidence without compromising the additive and auditable nature of the enrichment process.

11 Conclusion

The Raw Material Knowledge Engine establishes the first major data foundation for MAERII. It converts scattered information about Minor Forest Produce into a structured, evidence-backed, and graph-ready knowledge layer.

The module has limited independent impact as a standalone product, but it is strategically important because later MAERII modules depend on accurate supply-side intelligence. Market demand analysis, product recommendation, design enhancement, quality grading, and pricing all require a reliable understanding of raw materials, regions, skills, and product possibilities.

The current implementation is therefore best understood as a foundational intelligence layer. Its value increases as other MAERII modules begin using the enriched data and relationship graph for higher-level reasoning.